PLANT LEAF DISEASE CLASSIFICATION USING DEEP LEARNING:

A Comparative Study and Streamlit-Based Prototype

Abir Singh Sharma, Ashay Bodhane, Vedant Pokale, Yash Dharmik
Department of Electronics and Computer Science
Ramdeobaba University, Nagpur, Maharashtra, India

Abstract—Plant diseases reduce crop quality and yield, and timely identification can help limit losses. This work develops an image-based classifier for eight plant-leaf disease categories using PlantDoc (field-condition photographs) and PlantVillage (controlled images). Three TensorFlow/Keras architectures are implemented and compared: a custom convolutional neural network (CNN), MobileNetV2 and EfficientNetV2-B0. The training pipeline combines image augmentation, class weighting, label smoothing, early stopping and learning-rate scheduling; the transfer-learning models are trained in two stages. Evaluation uses test-time augmentation (TTA) and reports the two test sets separately. On the 76-image PlantDoc test set, MobileNetV2 achieves the highest accuracy (81.58%) and weighted F1-score (81.38%), followed by EfficientNetV2-B0 (78.95% accuracy) and the custom CNN (38.16%). On the 462-image PlantVillage test set, the corresponding accuracies are 98.92%, 98.70% and 85.28%. A Streamlit prototype uses the trained MobileNetV2 model to provide image-based predictions and confidence scores. The results highlight both the benefit of transfer learning and the remaining domain gap between controlled and field-condition imagery.

Keywords—*plant disease classification, deep learning, CNN, MobileNetV2, EfficientNetV2, transfer learning, PlantDoc, PlantVillage, Streamlit.*

I. INTRODUCTION

Agriculture supports the food supply and economy of many countries, and plant diseases are a persistent threat to crop quality and productivity. Disease identification has traditionally relied on manual inspection by farmers or agricultural experts. This is slow, and it can be unreliable when different diseases produce similar visual symptoms.

Advances in computer vision allow disease symptoms to be recognised automatically from leaf photographs. Convolutional neural networks (CNNs) learn visual features directly from images and are the standard approach for this task. A recurring difficulty, however, is that models trained on clean laboratory images often perform much worse on photographs taken in real fields, where backgrounds, lighting and image quality vary.

This project addresses that difficulty by training on both PlantDoc and PlantVillage while reporting results on each test set separately. The main objectives are:

- Build an automated classifier for eight plant-leaf disease categories.
- Compare a custom CNN with two pretrained architectures, MobileNetV2 and EfficientNetV2-B0.
- Apply data augmentation, class weighting, label smoothing, transfer learning and test-time augmentation.
- Measure the gap between controlled (PlantVillage) and in-the-wild (PlantDoc) performance.
- Develop an interactive Streamlit web application around the trained model.

II. RELATED WORK AND BACKGROUND

PlantVillage. PlantVillage is an open-access repository of labelled images of healthy and diseased leaves captured under controlled conditions [2]. It is widely used for plant-disease research, but its uniform backgrounds and near-duplicate images of the same leaf can make results look better than they will be in practice.

PlantDoc. PlantDoc was built to support visual plant-disease detection on images from natural environments [1]. Its images vary in lighting, background and quality, which makes it a more realistic but also a harder benchmark.

MobileNetV2. MobileNetV2 is a lightweight CNN built from inverted residual blocks with linear bottlenecks, designed for devices with limited computing resources [3].

EfficientNetV2. EfficientNetV2 is a family of CNNs designed for faster training and better parameter efficiency while keeping strong accuracy [4].

III. DATASETS

A. PlantDoc

The PlantDoc repository provides separate train and test folders. Eight disease classes present in both folders were selected, giving 883 original training images and 76 test images. A 15% validation subset (135 images) was selected from the original training images using a random split with random state 42; the notebook does not specify stratification. The original test folder was left untouched. The resulting split contains 748 training, 135 validation and 76 test images.

B. PlantVillage

The same eight diseases were extracted from PlantVillage using the TensorFlow Datasets loader, with at most 400 images kept per class. This gives 3,075 images in total (Apple Cedar Rust has only 275 available). An 85/15 split gives 2,613 training and 462 test images.

C. Selected Disease Classes

TABLE I. Selected classes and image counts

No.	Disease class	PlantDoc train*	PlantDoc test	PlantVillage
1	Apple Scab Leaf	83	10	400
2	Apple Rust Leaf	79	10	275
3	Bell Pepper Leaf Spot	62	9	400
4	Corn Leaf Blight	180	12	400
5	Corn Rust Leaf	106	10	400
6	Potato Leaf Early Blight	109	8	400
7	Squash Powdery Mildew Leaf	124	6	400
8	Tomato Septoria Leaf Spot	140	11	400
	Total	**883**	**76**	**3,075**

**Original PlantDoc training images before the 15% validation split (883 = 748 training + 135 validation).*

TABLE II. Final data splits used in the experiments

Split	PlantDoc	PlantVillage	Used for
Training	748	2,613	Model fitting
Validation	135	—	Early stopping, model selection
Test	76	462	Final evaluation

Both training sets are combined. Because PlantDoc is the real-world target but is small, each PlantDoc training image is repeated three times per epoch (augmentation makes every pass different), giving 2,613 + 3 × 748 = 4,857 training samples per epoch. Validation uses PlantDoc images only, so that model selection optimises for the realistic domain.

IV. METHODOLOGY

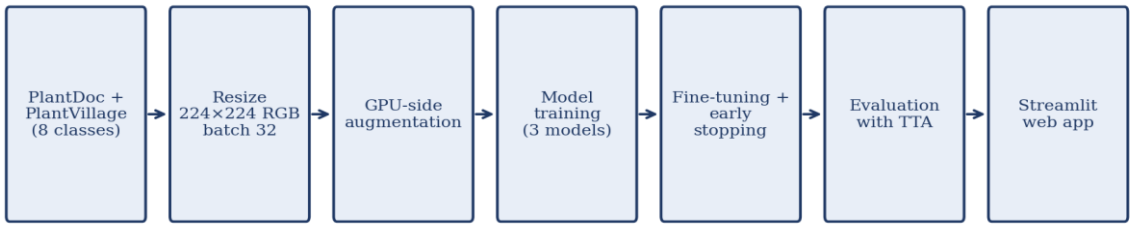


Fig. 1. Data-to-deployment pipeline.

A. Preprocessing

All images are decoded as RGB, resized to 224 × 224 pixels, cached in memory as 8-bit integers and processed in batches of 32 with prefetching. A fixed random seed of 42 is used throughout for reproducibility. Mixed-precision (float16) training is enabled when a GPU is available.

B. Data Augmentation

Augmentation runs on the GPU as the first layers of each model and is switched off automatically at inference time. It consists of horizontal and vertical flips, random rotation (±15%), random zoom (20%), random translation (10% in each direction), random contrast (20%) and random brightness (15%).

C. Loss Function and Class Imbalance

Categorical cross-entropy with label smoothing of 0.1 is used as the loss. Balanced class weights are computed over the effective training distribution (after PlantDoc repetition) and range from 0.76 (Corn Leaf Blight) to 1.40 (Apple Rust Leaf).

V. MODEL ARCHITECTURES

A. Custom CNN

The baseline has four convolutional blocks with 32, 64, 128 and 256 filters. Each block contains two 3 × 3 convolutions, each followed by batch normalisation and ReLU, then max pooling and dropout (0.1). The features pass through global average pooling, dropout (0.4), a dense layer of 128 ReLU units, dropout (0.3) and an eight-way softmax. The model has 1,209,064 parameters (1,207,144 trainable). It is trained with Adam at a learning rate of 0.001 for up to 40 epochs.

B. MobileNetV2

An ImageNet-pretrained MobileNetV2 backbone is used with inputs rescaled to [−1, 1]. The classification head is global average pooling, dropout (0.3), a dense layer of 256 ReLU units, dropout (0.3) and an eight-way softmax. In stage 1 the backbone is frozen and only the head is trained (up to 12 epochs, learning rate 0.001). In stage 2 the last 60 backbone layers are unfrozen and fine-tuned (up to 25 epochs, learning rate 0.00002), with batch-normalisation layers kept frozen.

C. EfficientNetV2-B0

EfficientNetV2-B0 uses the same head and the same two-stage schedule as MobileNetV2. It rescales raw 0–255 inputs internally, and the last 100 backbone layers are unfrozen during fine-tuning, with batch-normalisation layers kept frozen.

D. Training Controls and Test-Time Augmentation

Early stopping on validation loss (restoring the best weights), learning-rate reduction on plateau (factor 0.3) and model checkpointing are used. For evaluation, TTA averages the predicted probabilities over four versions of each image: the original, a horizontal flip, a vertical flip, and both flips. The experiments were run in TensorFlow 2.21 on a GPU-enabled Google Colab runtime.

E. Evaluation Metrics

Accuracy, and weighted precision, recall and F1-score are computed with scikit-learn. Per-class precision, recall and F1-score are also reported for the PlantDoc test set.

VI. RESULTS

A. Custom CNN

Table III gives the accuracy of the custom CNN on both test sets, with and without TTA.

TABLE III. Custom CNN accuracy on the two test sets

Test set	Images	Without TTA	With TTA
PlantDoc test	76	38.16%	38.16%
PlantVillage test	462	85.50%	85.28%

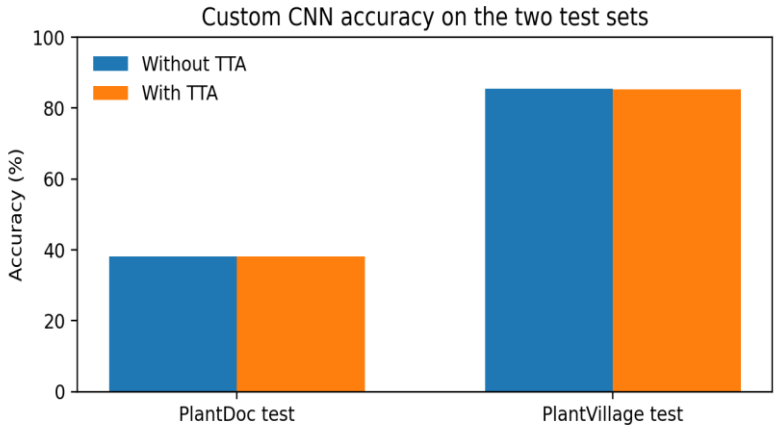


Fig. 2. Custom CNN accuracy on the PlantDoc and PlantVillage test sets.

For the updated notebook run, the custom CNN obtains 38.16% accuracy on PlantDoc with and without TTA. On PlantVillage, its accuracy changes from 85.50% without TTA to 85.28% with TTA. The

PlantDoc weighted precision, recall and F1-score with TTA are 32.46%, 38.16% and 32.79%, respectively. The PlantDoc test set contains only 76 images, so one prediction changes accuracy by approximately 1.32 percentage points. The small difference between the two PlantVillage TTA settings is likewise not evidence of a meaningful performance change.

B. Per-Class Performance on PlantDoc

TABLE IV. Custom CNN per-class results on PlantDoc test (with TTA)

Class	Precision	Recall	F1-score	Support
Apple Scab Leaf	0.25	0.20	0.22	10
Apple Rust Leaf	0.20	0.10	0.13	10
Bell Pepper Leaf Spot	0.00	0.00	0.00	9
Corn Leaf Blight	0.64	0.58	0.61	12
Corn Rust Leaf	0.47	0.70	0.56	10
Potato Leaf Early Blight	0.28	0.88	0.42	8
Squash Powdery Mildew Leaf	0.57	0.67	0.62	6
Tomato Septoria Leaf Spot	0.20	0.09	0.12	11
Macro average	0.40	0.33	0.34	76
Weighted average	0.32	0.38	0.33	76

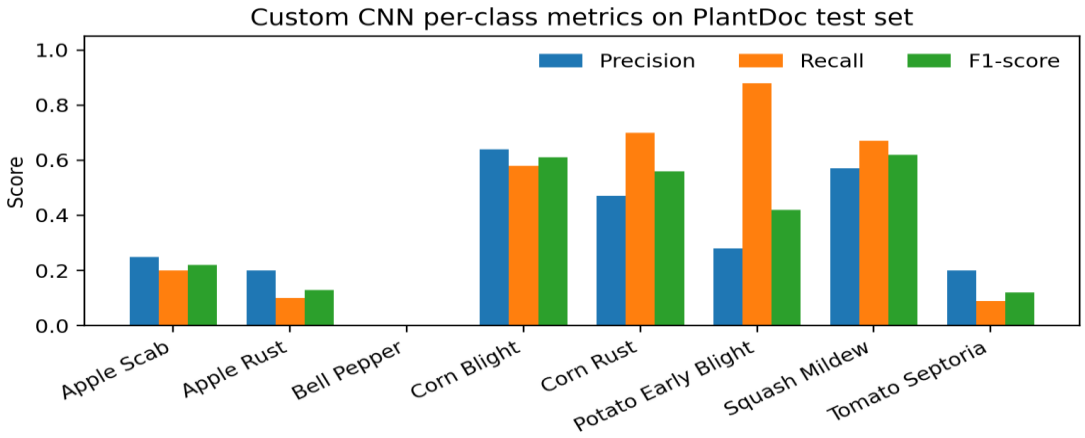


Fig. 3. Per-class precision, recall and F1-score of the custom CNN on the PlantDoc test set.

The updated CNN results remain uneven across classes. Corn Leaf Blight has the highest class F1-score (0.61), followed by Corn Rust Leaf (0.56) and Potato Leaf Early Blight (0.42). Bell Pepper Leaf Spot has zero precision, recall and F1-score in this run, while Tomato Septoria Leaf Spot has an F1-score of 0.12. Apple Scab Leaf and Apple Rust Leaf also remain difficult, with F1-scores of 0.22 and 0.13. These results indicate that the custom CNN has difficulty learning robust class-specific features from the available training data, particularly for visually similar symptoms and classes with limited samples.

C. Model Comparison

Table V compares the three architectures using the notebook's TTA evaluation. MobileNetV2 achieves the highest PlantDoc test accuracy, weighted F1-score and PlantDoc validation accuracy. EfficientNetV2-B0 has the highest weighted precision by a small margin, but lower PlantDoc recall and accuracy.

TABLE V. Comparison of the three architectures (with TTA)

Model	PlantDoc acc.	PlantVillage acc.	PlantDoc F1 (wtd.)	PlantDoc val. acc.
Custom CNN	38.16%	85.28%	32.79%	58.52%
MobileNetV2	81.58%	98.92%	81.38%	82.22%
EfficientNetV2-B0	78.95%	98.70%	79.07%	78.52%

D. Discussion

The principal result is the improvement obtained through transfer learning on the primary PlantDoc evaluation set. MobileNetV2 reaches 81.58% accuracy, compared with 78.95% for EfficientNetV2-B0 and 38.16% for the custom CNN. Its weighted F1-score is 81.38%, while EfficientNetV2-B0 records 79.07% and the custom CNN 32.79%. The PlantVillage results are higher for all three models (85.28%–98.92%), but should be interpreted as controlled-domain performance rather than a direct estimate of field performance. The gap between PlantVillage and PlantDoc accuracy is 47.12 percentage points for the custom CNN, 17.34 points for MobileNetV2 and 19.75 points for EfficientNetV2-B0. This pattern is consistent with the greater visual variability of field-condition images.

The results support MobileNetV2 as the strongest model among those evaluated in this experiment. Its PlantDoc validation accuracy (82.22%) is also the highest of the three models, which is consistent with its final test-set performance. Nevertheless, the PlantDoc test set is small, and the results should be treated as an evaluation of this prototype rather than proof of general performance across crops, regions, cameras or growing conditions.

VII. STREAMLIT WEB APPLICATION

The notebook includes a single-image prediction demonstration, while the GitHub repository contains a separate Streamlit application (app.py). The repository is configured with best_mobilenetv2.keras, so the deployed application code uses MobileNetV2. This is consistent with the notebook comparison, in which MobileNetV2 has the highest PlantDoc validation accuracy (82.22%) and test accuracy (81.58%). The repository contains four files:

TABLE VI. Application repository files

File	Purpose
app.py	Streamlit application
best_mobilenetv2.keras	Trained MobileNetV2 model used for deployment
class_names.json	Eight disease class labels
requirements.txt	Python dependencies

The application works as follows:

1. The user opens the web application and uploads a leaf image.
2. The image is converted to RGB and resized to 224 × 224 pixels.
3. Test-time augmentation produces four versions of the image.
4. The model predicts class probabilities for each version, and the four predictions are averaged.
5. The class with the highest averaged probability is displayed together with its confidence score.

The confidence score is the model's output probability. It is not a guarantee of correctness, and the model can give confident answers for images outside its eight supported classes.

Deployment note. A cloud deployment attempt reported that TensorFlow CPU 2.19.0 had no compatible installation for the Python 3.14 runtime used by the build environment. This is a dependency/runtime compatibility issue. The environment must use a Python version supported by the selected TensorFlow release before the application can be rebuilt reliably. This paper describes the Streamlit application and repository implementation; it does not claim that a public cloud deployment is currently running. Source code: https://github.com/abirsinghsharma/Plant-Disease-Classification.

VIII. LIMITATIONS AND FUTURE SCOPE

A. Limitations

1. The PlantDoc test set has only 76 images, so accuracy estimates are sensitive to individual predictions and small differences between models are within noise.
2. PlantVillage accuracy is optimistic because of controlled imaging and near-duplicate leaves.
3. Only eight disease classes are supported; other plants, diseases or unusual backgrounds can be misclassified.
4. The model has no unknown-class detection, so it may give confident but wrong answers on unsupported inputs.
5. Several classes have few training images, which leads to weak per-class results.
6. Cloud deployment requires matching Python and TensorFlow versions.

B. Future Scope

- Collect more field images and extend the set of plants and diseases.
- Report confidence intervals and repeated-seed experiments, and include per-class metrics and confusion matrices for every architecture.
- Add confidence calibration and unknown-class rejection.
- Use stricter, leaf-level splits for PlantVillage to remove near-duplicates.
- Optimise the model for mobile devices and integrate it with agricultural advisory services.

IX. CONCLUSION

This work developed an eight-class plant-leaf disease classification pipeline using PlantDoc and PlantVillage, and compared a custom CNN with MobileNetV2 and EfficientNetV2-B0. On the primary PlantDoc test set, MobileNetV2 achieved the strongest overall result, with 81.58% accuracy and 81.38% weighted F1-score. EfficientNetV2-B0 achieved 78.95% accuracy and 79.07% weighted F1-score, while the custom CNN achieved 38.16% accuracy and 32.79% weighted F1-score. On PlantVillage, the models achieved 98.92%, 98.70% and 85.28% accuracy, respectively. The difference between the two test domains reinforces the importance of evaluating plant-disease classifiers on field-condition images. A Streamlit prototype using MobileNetV2 demonstrates image upload and prediction, although cloud deployment requires a compatible Python and TensorFlow environment. The system should be regarded as a preliminary decision-support prototype, not a replacement for expert diagnosis. Future work should expand field datasets, validate results across repeated splits, calibrate confidence and introduce rejection of unsupported inputs.

REFERENCES

[1] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat and N. Batra, "PlantDoc: A Dataset for Visual Plant Disease Detection," in Proc. 7th ACM IKDD CoDS and 25th COMAD, 2020, pp. 249–253, doi: 10.1145/3371158.3371196.

[2] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2015.

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520.

[4] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in Proc. 38th Int. Conf. Machine Learning (ICML), PMLR vol. 139, 2021, pp. 10096–10106.

[5] TensorFlow, "TensorFlow and Keras documentation." [Online]. Available: https://www.tensorflow.org/

[6] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/